Quantitative Trading Model for Cryptocurrency Price Prediction: Deep Learning LSTM Implementation

Executive Summary

This project implements an advanced LSTM-based cryptocurrency trading system using high-frequency limit order book (LOB) data. **Dataset:** 631,263 samples, 35 engineered features across BTC, ETH, LINK, SOL, DOT, SHIB, DOGE. **Goal:** Predict 10-step ahead price movements with stable convergence across multiple assets.

Model Architecture

Network	LSTM(64) → LSTM(32) → Dense(1)
Regularization	Dropout(0.3), BatchNorm, EarlyStopping
Optimizer	Adam(lr=5×10^{-4}, β_1=0.9, β_2=0.999)
Loss Function	MSE + L2 (λ=10^{-3})
Split	Train/Val/Test = 60%/20%/20%
Sequence	10 timesteps in, 10-step forecast out

Feature Engineering (35 Features)

LOB (20): 4-level bid/ask prices & volumes, lagged prices t-1..t-4. **Microstructure (8):** Spread, mid-price, order imbalance, volume ratio, depth imbalance. **Technical (7):** Log returns, rolling volatility, MA(5/10/20), momentum, VWAP.

Data Processing Pipeline

1. Cleaning: IQR-based outlier clipping, negative volume removal. 2. Feature Engineering: Compute 35 metrics, handle division by zero. 3. Windowing: Sliding window (stride=1). 4. Scaling: StandardScaler for features, RobustScaler for targets. 5. Training: Multi-asset sequential training with checkpointing.

Performance & Results

Key Findings:

- Market microstructure features are strongest predictors.
- L2 + Dropout prevent overfitting.
- Model generalizes across assets.
- Engineered features outperform raw prices.

Implementation Details

Tech Stack: TensorFlow/Keras, scikit-learn, pandas, numpy. **Persistence:** `.h5` model, scalers (.pkl), plots (.png), predictions (.csv). **Production:** Real-time inference, automated retraining, robust error handling.

Mathematical Framework

$$\hat{P}_{t+h} = f_{\text{LSTM}}(X_{t-w:t}), \quad X_t \in \mathbb{R}^{35}$$

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(P_i - \hat{P}_i)^2 + \lambda\|\theta\|_2^2$$

$w = 10$, $h = 10$, $\lambda = 10^{-3}$.

Code Workflow & Key Components

Execution Flow:

1. Load raw CSV data for multiple assets.
2. Run `process_data()` to clean, engineer features, and scale.
3. Create time-series sequences via `create_sequences()`.
4. Build model with `create_model()`.
5. Train using `train_on_multiple_csv()` with checkpoints.
6. Evaluate with `test_model_csv()` and export predictions.

Important Functions:

- `process_data()`: Orchestrates full preprocessing pipeline.
- `calculate_features()`: Generates all 35 LOB, microstructure, and technical metrics.
- `create_model()`: Builds LSTM layers with regularization.
- `train_on_multiple_csv()`: Trains across assets sequentially, saving best weights.
- `test_model_csv()`: Loads model/scalers, produces forecasts.

Critical Code Aspects:

- Uses `ModelCheckpoint` to save best epoch weights.
- Standardizes inputs while robust-scaling targets to prevent outlier skew.
- Handles NaN/Inf values to prevent runtime errors.
- Uses small batch size (32) for stable LSTM convergence.
- Patience in early stopping prevents premature halt.

Future Enhancements

Architecture	Transformer models, attention
Features	Order flow toxicity, sentiment analysis
Training	Ensemble methods, transfer learning
Production	Real-time streaming, risk management
Optimization	Hyperparameter tuning, AutoML

Impact: Combines traditional quant finance with deep learning for production-ready crypto trading. Demonstrates SOTA regularization, cross-asset capability, and comprehensive feature engineering.